FARMWISEAI TASK 6 · CHENNAI DISTRICT

District IQ: study guide and demo script

Everything in this guide was checked against the code in `D:\farmwisenew` and the latest pipeline build (as of 2 Oct 2026, 12:32 IST). Where the old script in `docs/` no longer matches the code, this guide says so and gives you the correct line.

Suggested study order for tonight:

1. Fix these first
2. Memorise the pitch
3. Learn the diagram
4. Pipeline steps
5. Algorithms
6. Rehearse the script once aloud
7. Quiz yourself

Contents

BEFORE ANYTHING

Fix these first

The 2-minute pitch

How it fits together

STUDY, LAYER BY LAYER

1 · Data sources

2 · News pipeline

3 · Intelligence pipeline

4 · Storage

5 · Web application

6 · Ask District IQ

7 · AWS and scheduling

8 · Testing and limits

REFERENCE

Algorithm cheat sheet

Tech stack and why

Numbers to remember

Glossary

TOMORROW

Setup checklist

Demo script

Questions and answers

If something breaks

READ THIS BEFORE YOU REHEARSE

Fix these first

The script in `docs/District_IQ_Demo_Script.md` and the slides were written a few days ago. The code has moved on, and the numbers change every build because the data window rolls forward. If a reviewer opens the code or the evaluation report, these are the places you would be caught out.

TOPIC	OLD SCRIPT SAYS	CODE AND LATEST BUILD SAY
Daily filter	"Today from midnight"	**The last 24 hours, ending when the page is opened.** Opened at 10 AM, Daily covers yesterday 10 AM to now. Weekly, Monthly and Quarterly are the last 7, 30 and 90 days.
How often data refreshes	"Every day at 6 AM"	**Hourly.** All 8 jobs in `district_intel/config.yaml` are `every_minutes: 60`. Task Scheduler job `DistrictIntel` runs hourly and at sign-in. In the cloud, GitHub Actions calls the Lambdas hourly (news every 3 hours).
Linking accuracy	Precision 0.99, F1 0.88	**Precision 0.968, recall 0.761, F1 0.852** on the held-out half. Say "about 0.97 precision and 0.85 F1 in the latest build; it moves a little with each build because the data window rolls".
Link thresholds	Link above 0.85, review 0.40 to 0.85	0.85 / 0.40 are only the defaults in `config.yaml`. Each build **learns** the merge threshold that gives the best F1 on the training half, and sets review = merge − 0.25. Latest build: **merge 0.55, review 0.30**.
News incident filter	F1 0.63, up from 0.41	Latest: **F1 0.61** (precision 0.63, recall 0.59), against **0.54** with weak labels alone.
The 300 "hand labels"	"300 hand labels we made ourselves"	The CSV says `labeled_by = claude-draft` for all 300 and `reviewed_by` is empty. Say "300 articles chosen by stratified sampling and labelled against written rules; the labels were drafted with an AI assistant and are marked for our review." If you spot-check 30 tonight, you can say you reviewed a sample.
Story merges	"129 merges checked by hand, all correct"	Latest build made **135** same-language merges and 0 English–Tamil merges. Only say "checked by hand" if your team looked at them.
Grievance count	~8,100	**~16,600** grievance records (180-day window). Police ~12,400, PWD ~2,350.
Tests	24 + 88 + 186	**26 + 88 + 186 = 300**. All 300 passed today.
Bedrock model	"Nova Micro or Ministral" as the fast router	Code default is **Nova Lite** (`apac.amazon.nova-lite-v1:0`) for both the router and the composer. Nova Micro and Ministral 3B/8B are allowed and were tested, but are not the default.
AWS writes	"Durable writes go to DynamoDB through the store API"	**Not wired yet.** With `DATA_BACKEND=aws` the website *reads* the curated store from S3. Users, complaints and logins still use MySQL. 27 tables (~58k rows) were copied to DynamoDB and the store Lambda exists; connecting the website's writes is the next step.

TOPIC	OLD SCRIPT SAYS	CODE AND LATEST BUILD SAY
Two browser profiles	Needed for Collector and officer	Not needed. Each portal has its own cookie (`dcd_session_citizen` , `_officer` , `_collector`), so you can be signed in as all three in one browser, in separate tabs.

WHAT YOU ARE DEMOING

Your local site (`localhost:3000`) runs on **MySQL** (no `DATA_BACKEND` set) with **Bedrock** as the AI provider. The public link `web-production-760af.up.railway.app` is a **one-time copy** on Railway's MySQL; it is not refreshed hourly. Both responded today.

KNOW WHO BUILT WHAT

The department officer console started in a teammate's fork and the Ask District IQ assistant came from the `farmwiseai-main` code; both were merged into this repo on 1 Oct. If a reviewer asks "who did this part?", have each teammate's answer agreed in advance.

MEMORISE THIS

The 2-minute pitch

Problem. A District Collector gets information from many channels: citizen grievances, police, PWD, hospitals, IMD weather, pollution data, the flood monitor and the news. Each has its own format and its own IDs, and none of them link. One flooded street shows up as four unrelated records. The Collector cannot see the real scale, departments do the same work twice, and a "closed" complaint carries no proof that it was fixed.

What we built. District IQ turns those disconnected records into one picture of real incidents. It runs in six steps:

1. **Collect**: 8 feeds, refreshed hourly. Real public data (news, IMD, CPCB, flood monitor, market prices) and realistic synthetic department data (grievances, police, PWD, hospitals).
2. **Understand**: a Python pipeline cleans, classifies, places every record in one of the 200 GCC wards, removes duplicates and links the same incident across sources.
3. **Store**: one curated store (SQLite, copied to MySQL; in the AWS version, S3 and DynamoDB).
4. **Show**: a citizen portal, a 4-page Collector console and 25 department officer consoles.
5. **Ask**: an AI assistant that answers in English, Tamil or Tanglish, and whose numbers are checked against the data.
6. **Decide**: the officer completes work with a site photo, the Collector verifies or sends it back, and every action goes to an audit log.

Why it is better. One incident instead of four records. Numbers come from SQL and statistics, never from the language model. Every priority comes with plain-language reasons. The loop is closed with proof. It handles Tamil. It is tested (300 automated tests) and it runs.

ONE SENTENCE TO REMEMBER

"Many disconnected records in, one trustworthy picture of real incidents out, with a reason for every priority and proof for every closure."

THE WHOLE SYSTEM ON ONE SCREEN

How it fits together

Each layer is one folder. A lower layer never depends on an upper one, so the collectors keep running if the website is down, and the dashboard can change without re-fetching any data.

Sources
outside world

Google News + publisher RSS, IMD, CPCB, Chennai Flood Monitor, AGMARKNET, GCC website; synthetic police, PWD, hospital, grievances

real | synthetic, flagged

↓ fetched hourly (Task Scheduler on the PC · GitHub Actions → Lambda in AWS)

Collectors
one folder each

Python (and Node.js for police). Each owns one domain, writes CSVs, keeps a run log, never changes for the dashboard's sake.

imd_weather_collector | cpcb_air_quality_collector | cfm_dss_collector | chennai_news_pipeline | *_generator

↓ CSVs read, never edited

district_intel
the brain, Python

load → classify → locate → overlay → dedup → link → incidents → analytics → agents. About 3.6 minutes on a laptop.

scikit-learn | SciPy | multilingual-e5 | pandas

↓ one build

Store
32 tables, 6 views

SQLite file `district_intel.db` → MySQL `district_intel` (pipeline-owned, swapped atomically) + `district_intel_ops` (people-owned, never overwritten). AWS copy: gzip JSON per table on S3.

↓ read by parameterised SQL

Web app
Next.js 14

Citizen portal · Collector console (4 pages) · 25 department officer consoles · Ask District IQ (Bedrock)

JWT cookies | Leaflet maps | ECharts | jsPDF

↓ officer sends work with a photo → Collector verifies or sends back

Decisions
accountability

`officer_reports`, `collector_decisions`, `audit_log` in `district_intel_ops`

Draw this from memory

If a reviewer says "explain the architecture", draw five boxes: **Sources** → **Collectors** → **district_intel** → **Store** → **Web app**, then an arrow back from the web app to the ops database for decisions, and a side box for AWS (Lambda collectors, S3 snapshot, DynamoDB, API Gateway, Bedrock).

LAYER 1

Data sources and collectors

Eight feeds. Each collector owns exactly one domain: the weather collector never writes water levels, the flood collector never writes rainfall. All are polite: about 1.5 seconds between requests to the same host, retries with backoff, `robots.txt` respected, size caps, and raw responses saved for audit. None of them bypasses a login or a CAPTCHA.

SOURCE	TYPE	HOW IT IS COLLECTED	WHAT IT GIVES
News `chennai_news_pipeline`	REAL	Google News RSS (English and Tamil, ~150 queries) + The Hindu, TOI, DT Next, New Indian Express, Dinamani RSS + Dinamalar sitemap. Full text with **trafilatura** where robots.txt allows.	~21,400 feed rows → ~9,700 unique articles. The only source showing what departments have *not* recorded.
IMD weather `imd_weather_collector`	REAL	IMD city-weather JSON API for 4 stations (Nungambakkam, Meenambakkam, Madhavaram, Ennore); district warnings from IMD's GeoServer WFS layer, colours from its WMS legend.	24-hour rainfall (ending 08:30 IST), temperature, humidity, 7-day forecast, district warning colour.
CPCB air quality `cpcb_air_quality_collector`	REAL	data.gov.in API if a key exists (none does), else **Playwright** reads the public CPCB map popups. History is behind a CAPTCHA, so we build our own history from our runs.	AQI, category, prominent pollutant per station.
Flood monitor `cfm_dss_collector`	REAL	Public JSON endpoints of the TN WRD Chennai Flood Monitoring DSS (chennaifloodmonitor.tn.gov.in).	River/canal gauge levels with warning and danger stages, lake storage, inflow, alerts, bulletins. 90-day rolling history.
Market prices (AGMARKNET)	REAL	AGMARKNET open APIs with an honest User-Agent (the daily report page is CAPTCHA-gated, so not used).	Vegetable prices at Chennai's Uzhavar Sandhai markets, weekly change.
GCC reference	REAL	Scripts read GCC's own grievance portal and who-is-who page.	17 complaint categories, 327 sub-types, 250 areas, 4,883 localities, 53,800 streets, 48 officials with phone numbers. 200 ward polygons from DataMeet (community data, not an official GCC feed).

SOURCE	TYPE	HOW IT IS COLLECTED	WHAT IT GIVES
Grievances	SYNTHETIC	Node script in the portal (`generate-synthetic-grievances.js`), English, Tamil and Tanglish text, plus real complaints filed through the portal.	~16,600 complaints over 180 days, with a duplicate truth file.
Police `police_dataset_generator`	SYNTHETIC	Node.js + TypeScript. 16 taluks, 60 stations, hour-of-day patterns per crime type, ~20% of events reported 2–4 times with jittered GPS/time/text.	~12,400 reports + `ground_truth_events.csv` to test dedup.
PWD `pwd_dataset_generator`	SYNTHETIC	Python standard library only. Each day has its own seeded random generator, so a day's rows never change between runs; lake storage simulated day by day.	Flood incidents, lakes, works, tasks, announcements.
Hospitals `chennai_hospital_data`	SYNTHETIC	Python; values for each (date, hospital) seeded with SHA-256, so they are reproducible in any run order.	Beds, occupancy, OPD, emergency cases, medicine stock, alert level.

The shared rain calendar (a problem you solved)

Each synthetic generator invented its own rainy days, so police said it rained on one day and PWD on another. Their flood days barely matched (daily correlation as low as 0.12). `dintel/sync.py` builds one calendar from the police generator's heavy-rain days plus IMD warning days and passes it to the other generators through their own options (`--rain-days` , `--rainfall-csv`). Now all sources agree on the weather, which is what makes cross-source linking testable.

WHY SYNTHETIC DATA AT ALL

Police, PWD, hospitals and the grievance back office do not publish open data. We generate realistic records, flag every synthetic row (`is_synthetic = 1`), and say so on stage. A real system plugs in by writing one new loader; nothing downstream changes.

Words to use

- **trustore**: this PC's antivirus re-signs HTTPS traffic; truststore makes Python check certificates against the Windows certificate store, so verification stays on.
- **Atomic write**: write to a temp file, then rename, so a crash never leaves a half-written CSV.
- **Rolling window**: collectors keep 90 days (generators 180); older rows drop off each run.

LAYER 2

News pipeline

1

Fetch

First run: a 90-day backfill. After that: the last 48 hours each run. Google News links are decoded offline when possible; Google's own article-resolver paths are disallowed by its robots.txt, so we don't call them.

2 Clean

HTML stripped, Unicode normalised, dates converted to IST, language detected (**langdetect**).

3 Chennai relevance filter

A **gazetteer** (place list) in `config.yaml`: district names, 17 revenue taluks (including Kolathur, which the original brief missed), 15 GCC zones and ~50 localities, in English and Tamil script. English matches whole words; Tamil matches word starts with case suffixes allowed (சென்னையில் matches Chennai). Ambiguous names (Ashok Nagar is also in Delhi) need context. Phrases like "Chennai Super Kings" are removed before matching.

4 Department and complaint tag

Weighted English and Tamil keyword lists put each article into one of 15 GCC departments. A department needs one strong term or two different weak terms; title words count double. An LLM layer exists but only runs if an OpenAI key is set (none is).

Why keywords, not an AI per article? Free, instant for 20,000 articles, deterministic and testable (88 tests). If a place is missing, add one line and run `--reprocess`; nothing is re-fetched.

LAYER 3 · THE HEART OF THE PROJECT

The intelligence pipeline (district_intel)

One command, `python run_pipeline.py build`, reads every collector's output and builds the curated store. Latest run: 218 seconds. The order is in `dintel/pipeline.py`:

```
load (8 sources) → classify → locate → overlay → dedup → link → incidents → analytics → agents
→ store
```

1 Load

WHAT One loader per source (`dintel/loaders/`) maps everything into one **70-column `events` table**: time, text, category, location, severity, status, and the source file it came from.

PRIVACY Names, emails and addresses are not copied. The mobile number becomes an **HMAC keyed hash** so repeat complainants can be counted without identifying them.

2 Classify categories

WHAT Structured sources use crosswalk tables (`reference/category_map.yaml`). Free text ("Other" complaints, news) goes to a classifier.

HOW **TF-IDF on character n-grams (2–5 letters) + Logistic Regression**, trained on grievance, police and PWD texts that already have a category. 30 classes. If the model is less than 45% sure, the record stays "Other".

WHY Character pieces match across spellings ("drainage", "drainge") and scripts (Tamil, Tanglish) without translation. Small, CPU-only, explainable.

SCORE Macro F1 0.983 on a held-out 20%. Admit why it is so high: synthetic complaint text comes from templates. Expect lower on real text.

3 News: stories and the incident filter

STORIES The same event is rewritten by many outlets. Step 1: headlines with character n-gram cosine ≥ 0.55 within 48 hours join one story. Step 2: **multilingual-e5-small** sentence embeddings join rephrased headlines at cosine ≥ 0.95, same category, within 24 hours; a story never grows past 6 articles.

TYPE Regex rules mark power-cut schedules, court cases, announcements, politics, sport and business as non-incidents.

FILTER Logistic regression on character n-grams + embeddings, trained on **weak labels** (keyword rules) plus 300 labelled articles (weighted 3×). Threshold chosen by 5-fold cross-validation. Latest: F1 0.61 vs 0.54 with weak labels alone.

FUNNEL 21,416 feed rows → 9,930 unique URLs → 9,620 documents → 7,032 stories → 1,678 incident articles → 886 news incidents.

LIMIT English–Tamil merging is switched off: from headlines alone, unrelated pairs scored as high as same-event pairs. A wrong merge hides an incident; a missed merge only double-counts, so we chose strict.

4 Locate (geography)

WHAT Every record is placed in one of **200 GCC ward polygons** by a point-in-polygon test, then ward → zone → taluk.

HOW	`matplotlib.path` with a bounding-box prefilter (about 1 second for 20,000 points). Points within 500 m outside a ward are snapped in and flagged. Ward → taluk is a majority vote of police/PWD points that carry a taluk code. Records with only a street name get that street's location, with its precision stored.
WHY	No GEOS/GDAL install (a common pain on Windows with Shapely/PostGIS).

5 Scenario overlay (one shared world)

WHAT	`dintel/world.py` plants clearly flagged records (`is_overlay = 1`) so the separately generated sources describe one world: on each rain day every source shows a flood response, plus multi-source incidents (drain overflow, wall collapse, dengue cluster) and news-anchored incidents.
WHY	It writes a **ground-truth file** saying which world event each record belongs to. That is how we can measure linking accuracy at all. The linker never reads it.
SIZE	800 planted records, 217 world events, 13 world rain days in the latest build. Real sources (news, IMD, CPCB, CFM) are never changed.

6 Duplicates within one source

WHAT	Five citizens reporting one pothole become one group.
HOW	**Blocking** (same category, nearby, close in time) + **KD-tree** for the distance test + **union-find** to build groups. Grievances: same type within 150 m and 72 h (or same street and ward), plus 569 officer "Duplicate of" decisions. Police: same category within 3 h and 600 m from different channels. PWD: 48 h and 200 m.
SCORE	B-cubed F1 0.996 for grievances and police against the generators' truth files.

7 Cross-source linking (the key algorithm)

PROBLEM	IDs never match across departments (police `CMP-…` , PWD `GRV-…` , portal `2026-NNNXXX`). So we link by **place, time and text**.
BLOCKING	Only compatible category groups (WATER, ROADS, CRIME…), within the category's own radius and time window (+48 h for news). A KD-tree finds nearby pairs, so the cost grows with neighbours, not with n^2.
FEATURES	Distance ÷ allowed radius, time gap ÷ window, text similarity (character n-gram cosine), same category, same ward, coarse location, involves news.
SCORER	**Logistic regression** trained on half the world events (split by a hash of the event ID); the other half is held out. Learned weights: distance −4.1, time gap −5.8, text +4.0, same category +2.7, same ward +1.2.
DECISION	Probability ≥ merge threshold → link. Review band → human review queue. Below → no link. Each record keeps only its best partner per other source. Union-find joins everything into incidents.
SCORE	Held-out: precision 0.968, recall 0.761, F1 0.852. 1,007 links, 302 pairs for review, 261 multi-source incidents.
WHY LR	Small data, CPU, and its weights are readable, so we can show *why* two records were linked. Uncertain pairs go to people.

8 Incidents, severity and priority

SEVERITY Facts of the event only (`dintel/severity.py`). Example: grievance about flooding +35, describes a hazard +15, affects school/hospital +8 each, repeat complaint +10. Police base by crime type (murder 70, road accident 35…), fatalities push to at least 70. Levels: **Severe ≥ 70, High ≥ 50, Medium ≥ 30, else Low**.

PRIORITY Computed at the build time ("as of"), so it never goes stale: severity + $5 \cdot \log_2$(sources) + $3 \cdot \log_2(1 +$ complaints) + 20 if over twice the deadline (10 if over) + 8 if Severe/High and unverified + 8 if 3+ new reports in 24 h + 5 each for vulnerable people, 2+ news outlets, only in the news, rain-linked, 3+ similar incidents here in 90 days. **Every bonus adds a plain-language reason.**

WHY LOG₂ Diminishing returns: one viral issue with 100 complaints cannot bury everything else.

DEADLINE Per category and severity, e.g. flooding [6, 24, 72, 120] hours for Severe/High/Medium/Low. Police cases are measured on first response, not closure.

ATTENTION Flags: Severe and not verified · well past deadline · needs several departments · only in the news · spreading (3+ reports in 24 h).

REAL EXAMPLE FROM THE STORE

`INC-20260919-0D73F1` , Lakes, bunds & sluices, priority 147.9: "Severe (98); reported by 3 sources; 4 citizen complaints; resolution over twice the 6 h target; affects hospital, worship; covered by 2 news outlets…"

9 Analytics

SPIKES Daily count per category × zone vs a **Poisson** baseline: the 28-day mean of earlier days × a weekday factor. A spike is $P(X \geq \text{observed}) < 0.01$ with at least 3 cases. 13 spikes in the last 7 days.

HOTSPOTS **DBSCAN** with haversine distance, eps 250 m, at least 5 incidents, per category, last 90 days. 323 clusters.

HOT WARDS **Getis-Ord Gi*** on 30-day counts with ward neighbours: hot when $z > 1.96$. 12 hot wards.

SENSORS **EWMA** (span 7) and a 28-day **z-score** for lakes, AQI, beds; $|z| \geq 3$ is unusual. Lake "days to full" from the 7-day slope.

KPIS Per period (1, 7, 30, 90 days) and zone, with the previous period for the change arrows.

10 Agents

Rule-based workflows that read the tables and write only drafts, alerts and review items; a person decides.

- **Data Steward**: source health, data-contract checks, quarantine, category drift (PSI, 7 days vs 28).
- **Action Planner**: drafts department tasks from playbooks for open Severe/High incidents with no actions.
- **Gap Finder**: incidents in the news that no department recorded (196 now). Feeds the "Only in news" card.
- **Linker**: turns review-band pairs into review items.
- **Watchdog**: turns spikes, sensor anomalies and IMD warnings into explained alerts.

- **Briefing**: fact pack from SQL → template text → **numeric verifier** (every number in the text must be in the fact pack). Latest build: 0 unverified numbers. An LLM rewrite is possible but off, and is accepted only if it passes the verifier.

THE PRINCIPLE TO REPEAT

Numbers come from SQL and statistics. A language model may word them, never invent them.

LAYER 4

Storage

SQLite (pipeline output)

`district_intel/output/district_intel.db`, ~160 MB, 32 tables + 6 views. Also written as CSVs and dashboard JSON. Zero setup, one file, easy to test.

MySQL 8 (what the website reads)

`python run_pipeline.py mysql` copies it after every build. Concurrent reads and writes for many users.

Two databases, one ownership rule

DATABASE	OWNER	ON EACH EXPORT
`district_intel`	pipeline	All 31 tables loaded into `*__new` tables, then swapped in with **one atomic** `RENAME TABLE`. Readers never see a half-loaded store. Row counts and per-column non-null counts are compared with SQLite; any mismatch fails the export.
`district_intel_ops`	people	Never dropped or truncated. Holds `collector_decisions`, `officer_reports`, `officer_steps`, `audit_log`, assistant chats, market prices, added sources.

A third database, `district_collector_dashboard`, is the citizen portal's own: users, complaints, status history, GCC reference data.

Main tables to name

`events` (one row per source record) · `incidents` (one per real-world incident, with priority and reasons) · `incident_members` and `incident_timeline` (the evidence) · `documents` (news articles) · `observations` (sensor readings) · `anomalies`, `hotspots`, `kpis` · `gaps`, `alerts`, `briefings`, `review_queue` · `ref_wards` etc.

LAYER 5

The web application

Next.js 14 (App Router) with React 18, TypeScript and Tailwind. Pages and REST API routes live in the same project: one deployment, shared types between front and back end. Database access is raw SQL through **mysql2** with `?` placeholders everywhere, which prevents SQL injection.

Sign-in and security

- Passwords hashed with **bcrypt**. After login the server issues a **JWT** (HS256, 7 days, library **jose**) in an **httpOnly cookie**, which page JavaScript cannot read, so it is safer against XSS than localStorage.
- One cookie per portal (citizen, officer, Collector), so all three can be signed in at once.
- `middleware.ts` checks on every request that the cookie's role matches the portal in the URL.
- Login rate limit: 20 attempts per IP and 8 per email per 15 minutes. Same error message for wrong email or wrong password.
- Inputs validated with **zod** schemas on the server. Aadhaar (if given) is encrypted with AES-256-GCM.
- Email OTP for registration and password reset (nodemailer). In development the OTP can be echoed in the API response; never in production.

Citizen portal

- **File a complaint**: GCC's own 17 categories and 327 sub-types; area → street from GCC's own lists; drop a pin on the map.
- The **ward comes from the pin** by point-in-polygon on the 200 ward boundaries, re-checked on the server. Never from PIN code or nearest centroid, which give confident wrong answers near ward edges.
- Map: **Leaflet + OpenStreetMap** (free, no key). Google Maps is optional if a key is set.
- "Others" complaints are classified by AI if a key exists, else by keywords; categories with no known department are flagged for manual routing.
- Complaint code like `2026-618SGP`; **Track complaints** shows full status history.

Collector console (`/collector`)

A fixed screen with no page scroll, designed on a 1366 × 680 canvas and scaled to the window. Top bar: logo, feed freshness chip, filters (Period, Zone, Taluk, Department), page arrows, search, alerts, export, workspaces, and Ask District IQ (Ctrl+K).

PAGE	WHAT IS ON IT
1 · Overview	Satellite map (Leaflet + Esri World Imagery) with zone/ward or taluk outlines, AQI stations, rain gauges, lakes, violet pins for added sources · District Snapshot strip · Open incidents by severity · **My Tasks** ("Closed work for you to check", with Send back / Close) · Today's Briefing · GCC contacts (zonal officer, department head).
2 · Briefing	The written briefing (read, download, save to a workspace) · **Developing stories** (news threads with 2+ articles) · follow-ups · **Only in news**: incidents in the news with no department record · added-source items.

PAGE	WHAT IS ON IT
3 · Trends	Open, past deadline, "officer says done, waiting for a check" · categories, last 30 days vs the 30 before · unusual spikes · recurring hotspots · unresolved by taluk. Clicking opens a detail view; it never filters.
4 · Environment & markets	Rainfall, air quality (with the satisfactory limit), reservoir storage. If a zone has no station, the nearest one is used and named with its distance · Uzhavar Sandhai vegetable prices per market.

Who gets on My Tasks (`FOR_COLLECTOR` in `src/lib/collector/intel.ts`): Severe or High, or 3+ citizen complaints, or several departments, or in the news, or Medium that took over twice its deadline. Reports sent by officers always come first. Everything else is left to department heads.

Incident pop-up: what happened, key facts, why it needs attention (only when it does), who is responsible (official's name from GCC), and "How it unfolded": grievances, police and news in one timeline. News outlets are counted by distinct publisher.

Exports: PDF briefing made in the browser (jsPDF + autotable, no server load) and a CSV action list (what needs you, plus closures to check).

Department officer console (`/officer`)

- 25 accounts (`officer.<code>@chennai.gov.in`), each with its own password. The department is read **from the database on every request**, never from the URL or the token.
- Page 1 Overview: the Collector's overview filtered to that department, plus **My Work**.
- Page 2 Work & insights: grievance board **New** → **In action** → **Sent to Collector** → **Verified**, plus department cards (police: crime and response; hospitals: beds and medicine; PWD: lakes and works; TNPCB: air; roads/electrical: rainfall).

The closed loop (show this live)

1 **Officer approves**

New → In action (`POST /api/officer/grievances/[id]` , step `approve`).

2 **Officer completes and sends**

Remarks + site photos (JPG/PNG/WEBP, checked by the file's first bytes so a renamed file cannot pass). Stored outside `public/` , served only to the Collector and that department. Saved in `officer_reports` .

3 **Collector checks**

The report is first in My Tasks. **Verify** closes it; **Send back** with a note returns it. Written to `collector_decisions` .

4 **Audit**

Every action lands in `audit_log` (who, when, which record, before and after).

LAYER 6

Ask District IQ (the AI assistant)

Collectors ask questions the fixed dashboard does not answer. An AI that invents numbers is dangerous in government, so the assistant is a guarded pipeline (`src/lib/assistant/pipeline.ts`):

1 **Spell correction**

Damerau-Levenshtein edit distance (a swapped pair of letters counts as one edit), only toward Chennai's own words (places, categories, vegetables). A real English word (274,000-word list) is never "corrected". The answer shows "Understood as…".

2 **Language**

English, Tamil script or Tanglish, detected by rules (share of Tamil letters + a romanised-Tamil word list such as *evlo, enna, irukku*). The reply follows the question's language.

3 **Guard (code, before any model)**

Refuses requests for citizens' personal details, requests to change or delete data, prompt-injection attempts and off-topic questions (sports, coding, other cities). Flags bulk requests ("export everything").

4 **Router**

A model picks which console tool answers. With no AI key, **rules pick the tool** ("Without AI" mode), so the demo never dies.

5 **Tools or a query plan**

For new questions the model fills a **JSON query plan** (table, measures, filters, time range) checked by zod. **It never writes SQL.** Our compiler turns the plan into one parameterised, read-only SELECT over a fixed catalogue of tables, with a 5-second limit and row caps. At most 3 queries per question; a bad plan is sent back for repair at most twice.

6 **Composer**

The model writes the sentence only from the facts that came back.

7 **Number verifier**

Every number in the answer and in chart titles must match a returned fact at the precision written ("97.7%" matches 97.65). Dates, IDs, ward numbers and window lengths are exempt. On failure: regenerate once, then fall back to a template.

8 **Chart check and save**

The **ECharts** spec is validated before display. The answer is saved with its plan and audited.

MODELS	Amazon Bedrock (Converse API), default **Nova Lite**, region ap-south-1. Each step forces a single tool call whose input must match a zod schema. Groq and OpenAI are optional fallbacks. At most 3 model calls per question.
OFFICERS	Answers are **locked to the officer's department on the server** (`lockTools` , `lockQuery`): ranking questions become their own figures; other departments' rows are dropped. The lock is in code, not in the prompt.
LIMITS	20 questions per minute, 300 per day per user, 1,500 characters per message, a daily token budget per user.
SEARCH	Semantic search over incidents with **multilingual-e5-base** running locally (transformers.js / ONNX): no key, no text leaves the machine.
TESTS	186 Vitest tests (answers, guards, typos, compiler, verifier, chart edits) + 35 golden questions in `eval/golden.json` .

AWS KEYS EXPIRE

Your Bedrock keys rotate about every 3 hours. Install a fresh one shortly before the review with `npm run ai:key` , then `npm run ai:check` . On the Railway site the token has probably expired, so it answers in "Without AI" mode. Say that openly if it happens.

LAYER 7

AWS deployment and scheduling

The challenge asked us to run on FarmwiseAI's AWS account (team 37, region **ap-south-1**, names `fai-tce-team37-*`) using only **S3, Lambda, API Gateway, DynamoDB, CloudWatch Logs and Bedrock**. The AWS version lives in a separate repo, `ApshanaSP/farmwise-aws` (local branch `aws-deploy`); the MySQL version is in `ApshanaSP/farmwise` .

SERVICE	WHAT WE USE IT FOR
Lambda	`collect-imd` , `-cfm` , `-pwd` , `-hospital` (and `-cpcb`): **one code package deployed several times**, an env variable `SOURCE` picks the collector. It pulls the previous CSVs from S3 into `/tmp` , runs the *unchanged* collector, pushes CSVs back, and writes to DynamoDB **only rows that changed**. A news Lambda (~9 min) returns 202 at once and keeps working by invoking itself. An `intel` Lambda hands out presigned S3 links; a `store` Lambda serves DynamoDB rows.
API Gateway	One HTTP API: `POST /refresh/<source>` , `/ingest/<source>` , `/intel/*` , `/store/*` . Every call needs a secret `x-refresh-key` header (compared in constant time). Throttled to 1 request/second, burst 5.
S3	Buckets `-raw` , `-uploads` , `-exports` . The curated store is saved as gzip JSON per table, **named by its SHA-256**, so unchanged tables are never re-uploaded. `intel/current.json` (the manifest) is written last, so readers switch from one complete build to the next. 31 tables, 19.4 MB.
DynamoDB	Tables `-intel` , `-portal` , `-summary` , `-user` with string `pk` / `sk` . Collector rows land in `-intel` . 27 portal tables (~58,000 rows) were copied into `-user` by `aws/migrate_mysql.py` , which is safe to re-run.

SERVICE	WHAT WE USE IT FOR
CloudWatch Logs	Each Lambda run's log: what it fetched, how many rows changed.
Bedrock	The assistant's models, inside AWS in Mumbai.

Three workarounds you should be able to explain

- **No EventBridge**: the team role is not allowed to create schedules. So **GitHub Actions** (`.github/workflows/aws-refresh.yml`) calls API Gateway on a cron: hourly at :15 UTC for IMD, flood, PWD, hospitals; every 3 hours for news.
- **CPCB and police run on the PC**: CPCB needs a real browser (Playwright) and police is Node.js; neither fits the Lambda package. `aws/push.py` sends their CSVs.
- **The PC still builds the intelligence store**: sentence-transformers is too big for a Lambda zip, and layers are not allowed. The PC builds, AWS stores. Moving the build to the cloud is Phase 2.

How the website uses AWS

With `DATA_BACKEND=aws`, the website downloads the S3 snapshot through the API and loads it into an **in-memory SQLite** database inside Node 22 (`node:sqlite`). A **dialect layer** (`src/lib/aws/dialect.ts`) rewrites our MySQL queries for SQLite (INTERVAL maths, IF, GROUP_CONCAT, integer division…), so the same queries run in both versions. Every 5 minutes it checks for a newer build and swaps the whole database at once. The website holds no AWS keys, only the API key.

SAY IT PRECISELY

"In the AWS version the dashboard reads the curated store from S3. Portal data has been migrated to DynamoDB and the store API exists; wiring the website's own writes to it is our next step." Don't claim the writes already go to DynamoDB.

Scheduling on the PC

Windows Task Scheduler job **DistrictIntel** runs `district_intel/scripts/run_intel.bat` hourly and at sign-in (it also runs on battery and catches up after the laptop was off). It runs the sources that are due (each retried 3 times), rebuilds only if an input changed, exports to MySQL, then uploads to S3 and pushes CPCB and police. A full cycle takes about 25–30 minutes, CPCB being the slowest (~11 min).

LAYER 8

Testing, accuracy and honest limits

SUITE	TOOL	TESTS	COVERS
Intelligence pipeline	pytest	26	Core algorithms, data readiness, MySQL export
News pipeline	pytest	88	Cleaning, relevance filter, classification, storage, URL canonicalisation

SUITE	TOOL	TESTS	COVERS
Website / assistant	Vitest	186	Answers, guards, typos, query compiler, number verifier, chart edits
Total		300	All passed on 2 Oct 2026. Plus 35 golden assistant questions.

Limits to say before anyone asks

- Department data is synthetic until real systems are connected. Linking accuracy is measured on planted, realistic-but-synthetic incidents; it has to be re-measured on real data.
- The news incident filter misses about 4 in 10 incident articles (recall 0.59). More labels are the fastest fix.
- English–Tamil story merging is off: headlines alone are not enough.
- 98% of news rows are headline snippets; only ~670 have full text.
- Rain and AQI history only grow from our own runs (official history is CAPTCHA-gated).
- Ward polygons are community data (DataMeet), not an official GCC publication.
- Text-only location is rough: from street names alone, ward accuracy is 0.46 and zone accuracy 0.78 (median error ~1 km). Pins are exact.
- The rate limiter is in-memory, fine for one server; several servers would need Redis.

REFERENCE

Algorithm cheat sheet

For each: what it does in one line, where it is used, the settings, and why we picked it over the obvious alternative.

ALGORITHM	PLAIN EXPLANATION	WHERE · SETTINGS	WHY THIS ONE
TF-IDF on character n-grams	Cut text into 2–5 letter pieces; weight pieces that are frequent in this text but rare overall.	Category classifier, news filter, link text similarity, story clustering	Handles typos, Tamil and Tanglish without translation; no GPU.
Logistic regression	Weighted sum of features → probability between 0 and 1.	Category classifier (30 classes), news incident filter, cross-source link scorer	Fast, CPU-only, weights are readable, so decisions can be explained. Small data.
Sentence embeddings (multilingual-e5-small)	A model turns a sentence into a vector; similar meaning → nearby vectors, even across English and Tamil.	News story merging (cosine ≥ 0.95, 24 h), news filter features	Free, offline, multilingual, runs on CPU; no data leaves the machine.
Cosine similarity	Angle between two vectors: 1 = same direction (same content).	Text similarity everywhere	Ignores text length.
Point-in-polygon	Is this point inside this ward's boundary?	Every record → one of 200 wards; 500 m snapping	Real boundaries, not nearest centroid; matplotlib.path avoids a GIS install.
KD-tree	A spatial index that finds "all points within r metres" quickly.	Dedup, linking, recurrence	Avoids comparing every pair (n^2).
Blocking	Only compare records that could plausibly match (same category group, near, close in time).	Dedup and linking	Cuts millions of comparisons to ~7,700 candidate pairs.
Union-find	Keeps groups; if A–B and B–C are joined, A, B, C are one group.	Duplicate groups, incidents, story clusters	Transitive, near-constant time per join, every member stays traceable.
Poisson test	How likely is today's count if the usual rate is λ? Very unlikely → spike.	Spikes: 28-day mean × weekday factor, $p < 0.01$, ≥ 3 cases	Correct for small counts, where mean + 2 SD raises false alarms.
DBSCAN	Groups points that have enough neighbours within a distance; the rest is noise.	Hotspots: haversine, eps 250 m, min 5, 90 days	No need to choose the number of clusters (K-means does); any shape; ignores scattered points; works in real metres.
Getis-Ord Gi*	Is this ward *and its neighbours* unusually high compared with the district?	Hot wards: 30-day counts, $z > 1.96$ (95%)	Standard spatial statistic; one busy ward alone is not a hot area.

ALGORITHM	PLAIN EXPLANATION	WHERE · SETTINGS	WHY THIS ONE
EWMA	Average where recent readings weigh more.	Sensors: span 7	Smooths noise but follows trends.
z-score	How many standard deviations today is from the 28-day mean.	Sensors: $\|z\| \geq 3$ is unusual	Simple, explainable.
$\log_2$ in priority	Each doubling adds the same points.	Sources and complaints bonuses	One viral issue cannot bury the rest.
B-cubed	Per record: how much of my predicted group is truly mine, and how much of my true group did I find.	Dedup evaluation	Standard for clustering quality.
PSI	Population Stability Index: how much the category mix shifted.	Data Steward: 7 days vs 28, > 0.25 = high	Standard drift measure.
Damerau-Levenshtein	Number of single-letter edits (incl. swapping two letters) between words.	Assistant spelling: "tomoto" → "tomato"	Typos are often swapped letters; corrects only toward district words.
HMAC hash	One-way keyed fingerprint.	Mobile numbers in the intel store	Count repeat complainants without storing the number.
Content addressing (SHA-256)	Name a file by the hash of its content.	S3 snapshot tables	Unchanged tables are never uploaded twice; easy caching.

REFERENCE

Tech stack and why

LAYER	TECHNOLOGY	WHY
Collection	Python 3.11, requests, pandas, BeautifulSoup, feedparser, trafilatura, langdetect, Playwright, truststore	Best data and scraping ecosystem; free, key-less feeds; secure HTTPS behind TLS inspection
Police generator	Node.js + TypeScript	Type errors caught before running
Intelligence	pandas, NumPy, scikit-learn, SciPy, sentence-transformers, matplotlib.path, PyYAML, pytest	Proven, explainable, CPU-only; thresholds in YAML, so tuning needs no code change
Stores	SQLite → MySQL 8 (mysql2); AWS: S3 + DynamoDB + in-memory node:sqlite	SQLite for the batch build; MySQL for concurrent web use; serverless on AWS (RDS not allowed)
Web	Next.js 14, React 18, TypeScript, Tailwind CSS	One full-stack app, one deploy, shared types

LAYER	TECHNOLOGY	WHY
Maps	Leaflet + OpenStreetMap, Esri World Imagery	Free, no API key or billing
Charts / PDF	ECharts; jsPDF + autotable	Rich charts; PDF made in the browser
Auth	bcryptjs, jose (JWT), zod, nodemailer	Standard and safe; jose works in both Node and Edge middleware
AI	Vercel AI SDK, AWS Bedrock Converse (Nova Lite), transformers.js	Switch providers by config; data stays in AWS; local embeddings
Cloud	Lambda, API Gateway, S3, DynamoDB, CloudWatch	Serverless, pay-per-use, allowed by the challenge
Scheduling	Windows Task Scheduler (PC), GitHub Actions (cloud)	EventBridge is denied to the team role
Hosting	Railway (web + MySQL)	A public link anyone can open

REFERENCE · LATEST BUILD, 2 OCT 2026

Numbers to remember

NUMBER	MEANING
8	refreshed feeds (news, IMD, CPCB, CFM, grievances, police, PWD, hospitals)
200 / 15 / 17	GCC wards / zones / revenue taluks
25	department officer consoles
32 + 6	tables + views in the curated store
~3.6 min	full pipeline build (218 s)
32,386	events (source records) → 27,435 incidents, 2,532 open
16,618 / 12,431 / 2,348	grievance / police / PWD records
21,416 → 9,620	news feed rows → unique documents (7,032 stories, 886 incident events)
0.968 / 0.761 / 0.852	linking precision / recall / F1 (held-out)
0.55 / 0.30	learned merge / review thresholds (defaults 0.85 / 0.40)
261	multi-source incidents
0.996	dedup B-cubed F1 (grievances and police)
0.983	category classifier macro F1 (template text, so optimistic)
0.61 (vs 0.54)	news incident filter F1 with labels (weak labels only)
300	labelled news articles (AI-drafted, to be reviewed)
0.95, 24 h	story embedding merge rule
250 m, 5	DBSCAN eps, min samples
p < 0.01, ≥ 3	spike rule
z > 1.96	hot ward (Gi*)
70 / 50 / 30	Severe / High / Medium severity cut-offs
13 / 323 / 12 / 196	spikes (7 days) / hotspots / hot wards / news-only gaps
27 tables, ~58k rows	copied to DynamoDB
31 tables, 19.4 MB	S3 snapshot (gzip)
300	automated tests (26 + 88 + 186), all passing

NUMBER	MEANING
`20 / 8 per 15 min`	login attempts per IP / per email
`20 / min, 300 / day`	assistant questions per user

REFERENCE

Glossary

PRECISION	Of the links we made, how many were right. 0.968 = about 3 wrong in 100.
RECALL	Of the true links, how many we found. 0.761 = we miss about 1 in 4.
F1	One number balancing precision and recall (their harmonic mean).
HELD-OUT / TEST SET	Data kept aside during training and used only to score the model, so the score is honest.
CROSS-VALIDATION	Split labels into 5 parts; train on 4, test on 1, five times; average.
WEAK LABELS	Labels made by rules (keywords) instead of people: many, noisy, free.
GROUND TRUTH	The known right answer, used only for scoring.
GAZETTEER	A list of place names with spellings, used to find places in text.
CROSSWALK	A table mapping one source's category names to ours.
AS OF	The moment the build's numbers are computed for (newest record time).
SLA / DEADLINE	Target hours to respond or resolve, per category and severity.
JWT	A signed token saying who you are and your role; the server checks the signature.
HTTPONLY COOKIE	A cookie the page's JavaScript cannot read, so an XSS script cannot steal it.
BCRYPT	A deliberately slow password hash, so stolen hashes are hard to crack.
PARAMETERISED SQL	Values sent separately from the SQL text (`?`), so input can never become code.
ATOMIC SWAP	Switch from old to new data in one step, so nobody sees half of each.
PRESIGNED URL	A temporary S3 link that allows one upload or download without AWS keys.
PROMPT INJECTION	Text that tries to make an AI ignore its instructions.
LAMBDA COLD START	Extra delay when a Lambda runs after being idle.
THROTTLING	A limit on requests per second.

TOMORROW, 30 MINUTES BEFORE

Setup checklist

- [ ] Laptop on charger, notifications off, browser zoom 100%, screen at least 1366 × 768.
- [ ] MySQL running; local site up: `npm run dev` in `chennai-grievance-portal-main`, open `localhost:3000`.
  Click every page once so Next.js compiles it; the first load in dev mode is slow.
- [ ] Fresh AWS key installed: `npm run ai:key`, then `npm run ai:check`.
  Do this as late as possible; keys last about 3 hours.
- [ ] Signed in as citizen, Collector and one officer, each in its own tab.
  Never show `officer-accounts.local.csv` or any `.env` on screen.
- [ ] One grievance in the officer's **New** tab ready for the closed-loop demo, and a small site photo (JPG/PNG) on the desktop.
- [ ] Each Ask District IQ question from the script tried once; drop any that answer badly.
- [ ] MySQL Workbench open with the demo queries pasted in a tab.
- [ ] VS Code open at `D:\farmwisenew`, folder tree visible, `district_intel/output/reports/evaluation.md` ready.
- [ ] AWS console tabs in ap-south-1: Lambda, API Gateway, S3, DynamoDB, CloudWatch Logs (filter `fai-tce-team37`).
- [ ] GitHub tab: `ApshanaSP/farmwise-aws` → Actions → AWS collector refresh runs.
- [ ] Live link tab: `web-production-760af.up.railway.app`.
- [ ] Backup screenshots of every console page in a folder.

Demo queries for MySQL Workbench (checked against the table definitions):

```
SELECT COUNT(*) FROM district_intel.events;

SELECT incident_id, category_label, source_count, priority_score, priority_reasons
FROM district_intel.incidents
WHERE is_open = 1
ORDER BY priority_score DESC
LIMIT 5;

SELECT at, actor, action, table_name, record_id
FROM district_intel_ops.audit_log
ORDER BY log_id DESC
LIMIT 5;
```

TOMORROW · ABOUT 25 MINUTES

Demo script

Follow the data: **where it comes from → how it is cleaned and linked → where it is stored → who uses it → how they act → the AI → the cloud**. Say this order at the start so reviewers always know where you are. Speak the **SAY** parts in your own words. **SCREEN** is what to click. **IF ASKED** is extra detail for questions.

SHORT ON TIME? THE 10-MINUTE VERSION

Opening (1) → pipeline in one minute (step 3, say only steps 1, 6, 7, 8) → Collector console page 1 and the incident pop-up → closed loop → two assistant questions (one normal, one refused) → AWS in one sentence → close.

1 · Opening `1 min`

SCREEN District IQ login page at `localhost:3000`.

SAY "Good morning. Our project is District IQ, the Chennai Collector's unified district intelligence system, built for FarmwiseAI Task 6."

"A Collector gets information from many channels: grievances, police, PWD, hospitals, IMD weather, pollution data, the flood monitor and the news. Each has its own format and IDs, and none of them link. One flooded street shows up as four unrelated records, so the Collector can't see the real scale, and a 'closed' complaint has no proof."

"District IQ turns those records into one picture of real incidents, tells the Collector what needs attention and why, routes the work to the right department, and tracks it to verified completion. I'll show it in the order the data flows: collection, cleaning and linking, storage, the three kinds of users, the AI assistant, and the cloud."

2 · The project layout `1 min`

SCREEN VS Code, folder tree of `D:\farmwisenew`.

SAY "The project has four layers, one folder each. The collectors, one folder per source. `district_intel`, our intelligence pipeline in Python. `chennai-grievance-portal-main`, the Next.js web app. And `aws`, the cloud deployment."

"A lower layer never depends on an upper one. The collectors keep running if the website is down, and we can redesign the dashboard without fetching any data again."

3 · Data collection `3 min`

SCREEN Open `imd_weather_collector/data/imd_weather_observations_chennai.csv` to show real rows. Then point at each collector folder.

SAY

"We collect eight feeds. Each collector owns exactly one domain and is never changed for the dashboard's needs; fixes happen later, in the pipeline."

"Real public data: Chennai news in English and Tamil, about 21,000 feed rows from Google News and publisher RSS; IMD weather from four Chennai stations with district warnings; CPCB air quality, read from the public map with Playwright; the Tamil Nadu Chennai Flood Monitor's river gauges and lake storage; and AGMARKNET vegetable prices."

"Police, PWD, hospital and back-office grievance data aren't published, so we generate realistic synthetic records: about 16,600 grievances, 12,400 police reports, PWD incidents and hospital figures over 180 days. Every synthetic row is flagged, and we always say so."

"The collectors are polite: they respect robots.txt, wait between requests, retry three times and never bypass a login or CAPTCHA. They run every hour."

"One problem we had to solve: each generator invented its own rainy days, so police and PWD disagreed about when it rained. We built a shared rain calendar from real IMD warnings and passed it to every generator. Now all sources agree on the weather, and that is what makes cross-source linking testable."

IF ASKED Python 3.11 for collection; truststore keeps HTTPS verification on behind antivirus TLS inspection; atomic file writes; 90-day rolling windows for collectors, 180 days for generators.

4 · News pipeline `1 min`

SCREEN `chennai_news_pipeline/config.yaml`, scroll to the gazetteer with English and Tamil place names.

SAY

"News needs extra work. We fetch a 90-day backfill once, then the last 48 hours each run. We clean it, then keep only Chennai news using this gazetteer of Chennai places in English and Tamil script, with spelling variants. Then weighted keyword lists tag each article with one of 15 GCC departments."

"We chose keywords over an AI call per article because it's free, instant for 20,000 articles and testable: this module has 88 tests. If a place is missing, we add one line and reprocess without fetching again."

5 · The intelligence pipeline `4 min`

SCREEN `district_intel/dintel/` folder, then `output/reports/evaluation.md`.

SAY

"This is the heart of District IQ. One command, `python run_pipeline.py build`, reads every collector's output and builds one curated store in about three and a half minutes."

"**Load**: eight loaders map every source into one events table. Mobile numbers become a keyed hash, so the store holds no phone numbers."

"**Classify**: free-text complaints go through a character n-gram TF-IDF and logistic regression classifier. Character pieces handle Tamil, Tanglish and typos. If it's under 45% sure, it says 'Other' instead of guessing."

"**News**: copies of the same story are merged, first by headline similarity, then with a multilingual embedding model at 0.95 cosine within 24 hours. A trained filter separates real incidents from power-cut schedules, politics and court news."

"**Locate**: every record goes into one of the 200 real GCC ward polygons with a point-in-polygon test, then ward to zone to taluk."

"**Duplicates**: five citizens reporting one pothole become one group, using blocking, a KD-tree and union-find."

"**Cross-source linking**, the key step. IDs never match across departments, so we link by place, time and text. A KD-tree finds nearby candidates; a logistic regression scores each pair on distance, time gap, text similarity, category and ward. High scores link, the uncertain band goes to a human review queue. On held-out events we get about 0.97 precision and 0.85 F1."

"**Incidents and priority**: each linked group becomes one incident with a priority score and plain reasons: past deadline, reported by three sources, affects a school, linked to rain. We use log2 so one viral issue can't bury everything else."

"**Analytics**: a Poisson test for unusual spikes, DBSCAN for recurring hotspots within 250 metres, Getis-Ord Gi* for hot wards, and EWMA for lakes, AQI and beds."

"**Agents** then write the outputs: data quality, action drafts, the Gap Finder for news no department recorded, alerts, and the briefing, where a verifier checks every number against the data. The rule throughout: numbers come from SQL and statistics, never from a language model."

IF ASKED The merge threshold is learned each build (latest 0.55, review 0.30), from defaults 0.85/0.40. Accuracy is measured on planted world events whose truth the linker never sees; we train on half and test on the other half.

6 · Storage `1.5 min`

SCREEN MySQL Workbench. Run the top-5 incidents query; point at `source_count` and `priority_reasons`. Then show `district_intel_ops` tables.

SAY "The pipeline writes one SQLite file, 32 tables and 6 views, and copies it into MySQL for the website."

"We use two databases with a strict ownership rule. `district_intel` belongs to the pipeline and is fully replaced each build: we load into new tables and swap them in with one atomic RENAME, so nobody sees a half-loaded dashboard. `district_intel_ops` belongs to people: the Collector's decisions, officer reports and the audit log. The pipeline never touches it."

"Here are today's top open incidents. This one was reported by three sources, and the reasons column says why it's on top."

7 · Citizen portal `2 min`

SCREEN Register page (don't submit), then the citizen tab → File a Complaint: pick type, area, street, drop the pin. Then Track Complaints.

SAY "The website is one Next.js application: pages and APIs in one project. Citizens register with an email OTP; passwords are hashed with bcrypt, and the session is a JWT in an httpOnly cookie that page scripts can't read. Middleware checks the role on every request."

"Complaint types are GCC's own 327 sub-types. When the citizen drops the pin, the ward is computed from the real ward boundary, and the server checks it again. The map is Leaflet with OpenStreetMap: free, no API key."

"The citizen can track every complaint and its status history. This complaint now flows into the pipeline like every other source."

8 · Collector console `5 min`

SCREEN Collector tab → `/collector`. Top bar, then page 1. Zoom and pan the map. Click an incident.

SAY "This is the Collector's console: one fixed screen, no scrolling, four pages."

"At the top, the feed chip shows how fresh the data is. The filters: period, where Daily means the last 24 hours up to now, then 7, 30 and 90 days; zone, taluk and department. A department is a filter on the same dashboard, not a separate page."

"Page 1: the satellite map with all 200 wards and 15 zones, the District Snapshot, open incidents by severity, Today's Briefing, and My Tasks: only what needs the Collector personally, with officers' completion reports first."

SCREEN The incident pop-up.

SAY "This is one real-world incident: what happened, key facts, why it needs attention, and who is responsible, with the official's name from the GCC website. 'How it unfolded' shows every record merged into it: the grievances, the police report, the news coverage. That's cross-source linking you can see."

SCREEN Page 2, then page 3 (click a spike), then page 4.

SAY "Page 2 is the briefing, developing news stories, and the card I like most: **Only in news**. These are incidents reported in the news that no department has recorded: the Collector's blind spot."

"Page 3, trends: categories over 30 days, unusual spikes from the Poisson model, recurring hotspots from DBSCAN, unresolved by taluk. Clicking explains the pattern: observed against expected."

"Page 4: rainfall, air quality and reservoir cards. If a zone has no station, we use the nearest and name it with its distance. And vegetable prices at each Uzhavar Sandhai."

SCREEN Export → PDF. Mention the CSV action list.

SAY "One click gives a PDF briefing for meetings, made in the browser. The CSV is an action list, not a raw dump."

9 · Officer console and the closed loop `3 min`

SCREEN Officer tab → `/officer`. Page 1, then page 2 → open a grievance in New → Approve → Complete & send: remarks + photo → Send to Collector.

SAY

"Each of the 25 departments has its own account. The department is read from the database on every request, never from the URL, so an officer can't see another department's data."

"Page 2 is the grievance board: New, In action, Sent to Collector, Verified, plus cards specific to this department."

"Let me close one. I approve it, so it moves to In action. The work is done, so I add remarks, upload the site photo as proof, and send it to the Collector. Photos are stored outside the public folder and shown only to the Collector and this department."

SCREEN Collector tab → refresh → My Tasks: the report is first → open → Verify (or Send back). Then the audit-log query in Workbench.

SAY

"On the Collector's side, the report is already first in My Tasks, with the remarks and the photo. The Collector verifies it, or sends it back with a note."

"And here is the new row in the audit log, written just now. That is the full loop: citizen, pipeline, officer, Collector, audit."

10 · Ask District IQ `3 min`

SCREEN Collector console → Ctrl+K. Ask, in order (use the ones that worked in rehearsal):

ASK	POINT TO MAKE
`Which zone needs attention now?`	Answer with a chart from real data
`make it a pie`	Follow-ups edit the last answer
`Velachery la indha week evlo accidents?`	Tanglish understood
`tomoto price at Anna Nagar`	Spelling: "Understood as tomato"
`Phone number of the citizen who filed complaint X`	Refused: personal data
`Delete all complaints`	Refused: read-only

SAY

"An AI that makes up numbers is dangerous in government, so the assistant is a guarded pipeline. First, plain code checks for personal-data requests, changes to data and prompt injection; that's why the last two were refused. Then a model on Amazon Bedrock picks the right console tool. For new questions it fills a structured plan. It never writes SQL: our compiler turns the plan into one read-only, parameterised query over a fixed list of tables, with a 5-second limit."

"The model writes the sentence only from the facts that came back, and a number verifier rejects any answer containing a number that isn't in the data. Without an AI key, rules take over, so it still answers."

SCREEN Optional: officer tab → Ask `Which department is slowest?`

SAY

"For an officer, answers are locked to their own department on the server. No prompt can get around it, because the lock is in the code, not the prompt."

11 · AWS `3 min`

SCREEN AWS console (ap-south-1): Lambda → API Gateway routes → GitHub Actions runs → CloudWatch log of a recent run → DynamoDB items → S3 `intel/current.json`.

SAY

"The challenge asked us to run on FarmwiseAI's AWS account with only S3, Lambda, API Gateway, DynamoDB, CloudWatch and Bedrock, so we built an AWS version in a separate repository."

"Our collectors run as Lambda functions: one code package deployed per source, wrapping our unchanged collectors. Each run writes only the rows that changed to DynamoDB. Everything goes through one API Gateway with a secret key and throttling; the website never holds AWS keys."

"Our AWS role isn't allowed to use EventBridge, so GitHub Actions calls the API every hour, and news every three hours. Here are the runs, and here's the log of one."

"The curated store goes to S3 as one compressed file per table, named by its hash, so unchanged tables are skipped, and a manifest is switched last. The website can load that snapshot into an in-memory database and run the same queries through a small SQL translation layer."

"The ML build still runs on our PC, because sentence-transformers is too big for a Lambda package. The PC builds, AWS stores. And the assistant runs on Bedrock, so its data stays in AWS in Mumbai."

IF ASKED Portal data (27 tables, ~58k rows) is migrated to DynamoDB and a store API exists; connecting the website's writes to it is the next step.

12 · Quality and limits `1 min`

SAY

"We measured it: 300 automated tests across the pipeline, news and website, plus a golden question set for the assistant. Linking at about 0.97 precision on held-out events, duplicates at 0.99, and zero unverified numbers in the briefings."

"And the limits: department data is synthetic until real systems are connected, so accuracy must be re-measured on real data; the news filter still misses about four in ten incident articles; and English–Tamil story merging is off because headlines alone weren't reliable. Each is on our roadmap."

13 · Close `30 s`

SAY

"To sum up: District IQ takes disconnected sources and turns them into one trustworthy picture of real incidents. It tells the Collector what needs attention and why, routes work to the right department, and tracks it to verified completion, with every number traceable to its source. Thank you. We're happy to take questions."

QUIZ YOURSELF

Questions and answers

Cover the answer, say yours aloud, then check.

THE IDEA

Q What problem does District IQ solve, in one sentence?

The Collector's information arrives in many disconnected systems, so one real incident appears as several unrelated records; District IQ links them into one incident, prioritises it with reasons, and tracks it to verified completion.

Q How is this different from a normal dashboard?

Most dashboards put sources side by side. We link the same real-world incident across grievance, police, PWD and news, explain every priority, find what only the news knows (the Gap Finder), and close the loop with photo proof and an audit log.

Q Who are the users?

Citizens (file and track complaints), the District Collector (whole district, verifies closures), and 25 department officers (their own department only).

Q Is the data real?

News, IMD, CPCB, the flood monitor, AGMARKNET prices, GCC complaint types, streets and officials are real. Police, PWD, hospitals and most grievances are synthetic because those departments don't publish open data. Every synthetic row is flagged; a real source plugs in with one new loader.

DATA AND PIPELINE

Q What is the difference between deduplication and linking?

Dedup works inside one source: five citizens reporting the same pothole. Linking works across sources: a complaint, a police report and a news story about the same event. Both end in union-find groups.

Q How do you link records when IDs don't match?

By place, time and text. Blocking and a KD-tree find nearby candidate pairs in compatible categories; a logistic regression scores each pair; above the learned threshold we link, the uncertain band goes to human review, and union-find builds the incident.

Q How did you measure linking accuracy without real ground truth?

The scenario overlay plants flagged multi-source incidents with realistic noise and writes which world event each record belongs to. The linker never sees that file. We split world events by hash: train on half, test on the other half. Latest: precision 0.968, F1 0.852.

Q **Isn't that circular, since you planted the data yourselves?**

Partly, yes, and we say so. It tests the algorithm against realistic noise (GPS jitter, time shifts, reworded text), and the truth is hidden from the linker, but it is not proof on real data. Phase 1 is connecting real department data and re-measuring.

Q **Why does the config say 0.85 but the build uses 0.55?**

0.85/0.40 are fallbacks. When ground truth is available, each build tests thresholds from 0.30 to 0.95 on the training half and keeps the one with the best F1; review starts 0.25 below it. The latest build chose 0.55 and 0.30.

Q **Why logistic regression and not deep learning?**

Small data, CPU only, and explainability: its weights show why two records were linked (distance −4.1, time gap −5.8, text +4.0). For government decisions that matters more than a few points of accuracy.

Q **Why character n-grams instead of words?**

They match pieces of words, so spelling variants, Tamil inflections and Tanglish look similar without any translation step.

Q **Your category classifier scores 0.98. Isn't that suspicious?**

Yes; it's high because synthetic complaint texts come from templates. We expect it to be lower on real text and would re-measure.

Q **How good is the news incident filter?**

F1 about 0.61 (precision 0.63, recall 0.59) with labels, against 0.54 from keyword rules alone. It still misses about 4 in 10 incident articles; more labels are the fastest improvement.

Q **Who labelled the 300 news articles?**

They were picked by stratified sampling and labelled against written rules (1 = a specific event a department must act on; 0 = schedules, politics, court, business…). The labels were drafted with an AI assistant and are marked for team review; scores use population-weighted 5-fold cross-validation.

Q **Why don't you merge English and Tamil stories?**

From headlines alone, unrelated English–Tamil pairs scored as high (0.86) as same-event pairs (0.84–0.98). A wrong merge hides an incident; a missed merge only double-counts. So the rule is strict and currently merges none.

Q **How is a record placed in a ward?**

Point-in-polygon against the 200 ward boundaries, snapping points within 500 m outside a ward (flagged). Without coordinates, the street or locality's known position is used, with its precision stored.

Q What is severity versus priority?

Severity uses only facts of the event (type, casualties, hazard, vulnerable people). Priority adds time and context at build time: deadlines, number of sources, complaints, growth, news coverage, rain, recurrence. So priority never goes stale.

Q Walk me through the priority formula.

Severity score, plus $5 \cdot \log_2(\text{sources})$, plus $3 \cdot \log_2(1 + \text{complaints})$, plus 20 if past twice the deadline (10 if past it), plus 8 if Severe/High and unverified, plus 8 if 3+ reports in 24 h, plus 5 each for vulnerable people, 2+ outlets, only-in-news, rain-linked, recurring. Each bonus adds a reason string.

Q Why Poisson for spikes?

Daily counts of incidents are counts of rare events, which Poisson models. For small numbers, mean + 2 SD gives false alarms. We use a 28-day baseline with a weekday factor, flag $p < 0.01$, and require at least 3 cases.

Q Why DBSCAN and not K-means?

K-means needs the number of clusters in advance and forces every point into one. DBSCAN finds dense areas of any shape, leaves scattered points as noise, and with haversine distance works in real metres (250 m ≈ a street block).

Q What is Getis-Ord Gi*?

A spatial statistic: a ward is "hot" only when it and its neighbours together are significantly above the district average ($z > 1.96$). One busy ward alone doesn't count.

Q What do the agents do, and are they LLMs?

They are rule-based workflows: Data Steward, Action Planner, Gap Finder, Linker, Watchdog, Briefing. They write drafts, alerts and review items, never change source records. The LLM option is off; if switched on, its text is accepted only after the numeric verifier passes.

Q How long does the pipeline take, and does it scale?

About 3.6 minutes for ~32,000 events on a laptop. Blocking and KD-trees keep comparisons near-linear instead of n^2. For a state, run one build per district.

Q What happens if a source fails?

Each job is retried 3 times; failures are logged in the refresh state. The last good data stays, the Data Steward marks the source degraded, and the console's feed chip shows the data's age.

WEBSITE AND SECURITY

Q Why Next.js?

One full-stack project: React pages and REST APIs together, one deployment, TypeScript types shared between front and back end, plus middleware for route protection.

Q How is login secured?

bcrypt password hashes; a JWT in an httpOnly cookie (one per portal); middleware checks the role per route; rate limits of 20 per IP and 8 per email per 15 minutes; the same error for wrong email or password.

Q Why a cookie and not localStorage for the token?

An httpOnly cookie can't be read by page JavaScript, so a cross-site-scripting bug can't steal the session.

Q How do you prevent SQL injection?

Every query uses placeholders with values bound separately. Filter values are checked with strict patterns (e.g. zone must be an integer 1–30). The assistant never writes SQL; it fills a plan that our compiler turns into bound, read-only queries.

Q Can an officer see another department's data?

No. The department comes from the database on every request, not the URL or token. Other departments' records return "not found", and the assistant is locked to that department in code.

Q How is personal data protected?

The intel store keeps no names, emails or phone numbers (mobile → keyed hash). Aadhaar is AES-256-GCM encrypted. Officer photos are served only through an access-checked route. The assistant refuses personal-data questions before any model runs.

Q How does the website get the ward of a complaint?

From the map pin, by point-in-polygon on the ward boundaries, re-checked on the server. Never from PIN code or the nearest centroid.

Q Why Leaflet and not Google Maps?

Free with OpenStreetMap, no API key, no billing. Google Maps is supported if a key is configured.

Q How does "Daily" work?

A rolling 24 hours ending when the page is opened. Opened at 5:30 PM, it covers yesterday 5:30 PM to now. Weekly, Monthly, Quarterly are the last 7, 30, 90 days.

Q What decides what goes on the Collector's My Tasks?

Severe or High, 3+ complaints, several departments, in the news, or Medium that took over twice its deadline. Officer completion reports always come first. Everything else stays with department heads.

AI ASSISTANT

Q Why not just use ChatGPT for everything?

Language models can produce confident wrong numbers. We use AI only to understand the question and word the answer. Every number comes from SQL, and a verifier rejects answers with numbers not in the data. Without AI the system still works.

Q How do you stop prompt injection?

A code guard runs before any model. The model never writes SQL, so it can't reach tables outside the catalogue. Queries run read-only. The officer lock is enforced in code.

Q Which model do you use and why?

Amazon Nova Lite on Bedrock in ap-south-1: it is on the challenge's allowed list, cheap and fast, and the data stays in AWS. The gateway can switch to Groq or OpenAI by configuration.

Q How does it understand Tanglish and typos?

Language is detected by rules (Tamil-letter share plus a romanised-Tamil word list). Typos are fixed by Damerau-Levenshtein distance, but only toward district words, and never when the word is real English; it shows "Understood as…".

Q What if the AI provider is down or the key expired?

Busy errors are retried with backoff, then the next provider. With no provider, rules route the question to a tool and a template writes the answer, labelled "Without AI".

AWS AND OPERATIONS

Q Why two versions, MySQL and AWS?

MySQL is simplest locally and on our public host. The challenge required AWS with only certain services and no RDS, so the AWS version uses S3, DynamoDB, Lambda and an in-memory SQLite in the website, with the same queries through a dialect layer.

Q Why GitHub Actions for scheduling?

EventBridge is denied to our team's AWS role. GitHub Actions calls our API Gateway on a cron with a secret key header, and it's free.

Q Why do CPCB and the ML build still run on the PC?

CPCB needs a real browser (Playwright) and the ML build needs sentence-transformers; both are too big for a Lambda package, and Lambda layers are not allowed. The PC builds and pushes; AWS stores.

Q What does a diff-only write mean?

Each Lambda compares this run's rows with the last run's and writes only new or changed rows to DynamoDB, so an hourly run costs a few writes, not thousands.

Q Does the website write to DynamoDB?

Not yet. The dashboard reads the curated store from S3. Portal data was migrated to DynamoDB and the store API exists; wiring the website's writes is the next step.

Q What does it cost to run?

Everything is pay-per-use: Lambda per run, DynamoDB per request, a few tens of MB on S3, a throttled API, Bedrock per token with a per-user daily budget. No always-on servers in the AWS version.

Q Is the live link up to date?

The Railway site is a one-time copy for sharing. The local system refreshes hourly.

FUTURE

Q Can this work for another district?

Yes. Wards, taluks, departments, categories, deadlines and playbooks are reference files (YAML/CSV), not code. A new district needs its reference data and source connectors.

Q What would you improve first?

Real department data, more human-reviewed news labels, full article text for English–Tamil merging, moving the ML build to the cloud, wiring website writes to DynamoDB, and learning priority weights from the Collector's past decisions.

STAY CALM

If something breaks

PROBLEM	WHAT TO DO AND SAY
Assistant says "Without AI" or "busy"	"Our AWS keys rotate every few hours; this is the rule-based fallback, so it still answers from the same data." Keep going.
Page loads slowly	Dev mode compiles each page on first visit. Say so, or switch to the Railway link.
Local site won't start	Use the Railway link for the web demo and screenshots for the rest.
Wi-Fi drops	Local site still works (MySQL is local); skip AWS and show screenshots.
Officer has nothing in New	Use a grievance in In action and go straight to Complete & send.

PROBLEM	WHAT TO DO AND SAY
Question you can't answer	"I'm not certain; I'll check that and get back to you." Then say what you do know about that part.
`.next` EUNKNOWN error in dev	Stop the server, delete `.next/server`, start again.